Analisi dei requisiti

Studio di Fattibilità (Prima dei Requisiti)

Obiettivo

Fase preliminare volta a stabilire **l'opportunità o meno** di realizzare un sistema software. Serve a capire se il progetto è conveniente, realizzabile e sostenibile prima di investire tempo e risorse nella definizione dei requisiti.

Aspetti Considerati

1. **Descrizione sommaria del sistema software**
   - Breve panoramica del sistema previsto.
   - Analisi delle **necessità degli utenti** e dei problemi che il software deve risolvere.
2. **Analisi di mercato**
   - Studio del mercato attuale e previsione del mercato futuro.
   - Valutazione del **costo di produzione** del software.
   - Stima della **redditività attesa dell'investimento**.
3. **Analisi tecnica di realizzabilità**
   - Identificazione degli **strumenti necessari** per lo sviluppo: software, librerie, framework, ecc.
   - Studio delle **soluzioni algoritmiche e architetturali** possibili per realizzare il sistema.
   - Valutazione dell'**hardware richiesto** per il corretto funzionamento del software.
   - Definizione del **processo di sviluppo**, che può includere:
     - Prototipazione rapida

    - Progetti esplorativi
    - Attività di ricerca per soluzioni innovative

Analisi dei Requisiti

Obiettivo

Studiare e definire il problema da risolvere per:

- Comprendere **cosa deve essere realizzato**.
- Documentare **cosa deve essere realizzato**.
- Consentire **negoziazioni tra committente e fornitore**.

Si tratta di un **processo di studio e analisi delle esigenze del committente e degli utenti**, con lo scopo di definire:

- Il **dominio del problema**.
- I **requisiti del sistema**.

> L'analisi dei requisiti si concentra sul cosa fare, non sul come farlo.

Attività principali

1. **Identificare i confini del sistema software**
    - Determinare il perimetro del sistema e il modo in cui interagirà con l'ambiente esterno.
2. **Definire e analizzare i requisiti di qualità**
    - Stabilire criteri come affidabilità, performance, sicurezza e usabilità.
3. **Negoziare le priorità con gli stakeholder**
    - Confrontarsi con clienti, utenti e manager per definire ciò che è più importante sviluppare per primo.
4. **Elaborare i requisiti in modo utile per il progetto**

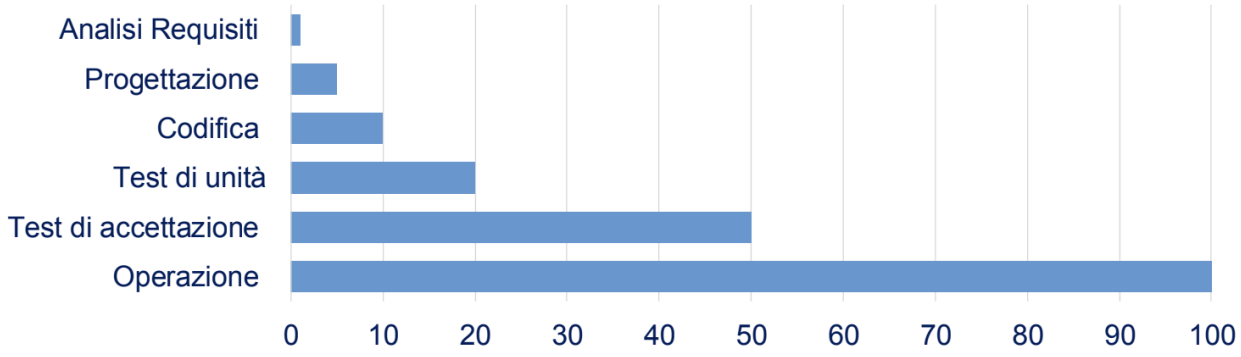
- Consentire ai **manager** di fare stime realistiche.
- Fornire agli **sviluppatori** informazioni chiare per progettazione, implementazione e test.

5. **Classificare i requisiti**
   - Suddividere le informazioni in categorie.
   - Assegnare requisiti ai sottosistemi pertinenti.
6. **Individuare e risolvere difetti nei requisiti**
   - Rilevare ambiguità, conflitti o requisiti incompleti.

Perché è importante

Secondo lo **Standish Group**, le principali cause di fallimento dei progetti software sono legate ai requisiti:

Fattore	% Risposte
Mancanza di input degli utenti	12.8
Requisiti e specifiche incomplete	12.3
Requisiti e specifiche in cambiamento	11.8
Mancanza di supporto esecutivo	7.5
Incompetenza tecnologica	7.0
Mancanza di risorse	6.4
Aspettative irrealistiche	5.9
Obiettivi poco chiari	5.3
Tempistiche irrealistiche	4.3
Nuove tecnologie	3.7
Altri fattori	23



Dominio

Acquisizione della conoscenza del dominio

Per definire il **dominio di applicazione** di un sistema software, si utilizzano strumenti e modelli che aiutano a comprendere e descrivere la realtà **indipendente dal software stesso**.

Glossario

- È una **collezione di definizioni** di termini rilevanti in un dominio specifico.
- Viene costruito dal **team di analisti** durante lo studio del dominio.
- Si **arricchisce progressivamente** man mano che emergono nuovi termini.
- Può essere **riutilizzato** in progetti successivi appartenenti allo stesso dominio.

Modelli

Per rappresentare il dominio si definiscono due tipi di modelli:

- **Modello statico**: descrive **entità** e **relazioni** (es. diagrammi concettuali, ER).
- **Modello dinamico**: descrive **processi** e **comportamenti** (es. diagrammi di attività, diagrammi di stato).

👉 La **conoscenza del dominio** evolve continuamente attraverso incontri e confronti con **committenti, utenti e stakeholder**.

Dominio: Istruzioni per l'uso

Quando si descrive o modella un dominio di applicazione di un sistema software, occorre concentrarsi su:

- **Entità**
- **Relazioni**
- **Processi**
- **Comportamenti**

> Questi elementi esistono indipendentemente dal sistema software che deve essere realizzato.

Il **sistema software**:

- opera all'interno di un dominio;
- può eventualmente **controllarlo o influenzarlo**;
- ma il **dominio esisteva già prima** dello sviluppo del software.

E i Requisiti?

Definizione

- I **software requirements** sono le **capacità** che il sistema deve fornire (dal libro di testo).

Secondo il **glossario IEEE** un requisito è:

1. Una **condizione o capacità necessaria a un utente** per risolvere un problema o raggiungere un obiettivo.
2. Una **condizione o capacità che un sistema deve soddisfare** per rispettare:
    - un contratto,
    - uno standard,
    - una specifica,
    - o altri documenti formali.

Requisiti Funzionali

Definizione

I **requisiti funzionali** descrivono le **funzionalità** che il sistema deve realizzare.

Si esprimono in termini di:

- **Azioni** che il sistema deve compiere.
- **Reazioni** del sistema software a specifici tipi di input.
- **Comportamenti** del sistema in situazioni particolari.

Esempi

- Il sistema software deve fornire un appropriato **visualizzatore** per i documenti memorizzati.
- L'utente deve essere in grado di **effettuare ricerche** sia sull'intero insieme di basi di dati sia su un sottoinsieme.
- Ad ogni nuovo **ordine** deve essere associato un **identificatore unico (Order_ID)**.

👉 Tradizionalmente, ai requisiti funzionali è dato il **maggior valore**, perché definiscono le funzionalità centrali del sistema.

Requisiti Non-Funzionali

Definizione

I **requisiti non-funzionali** descrivono le **proprietà del sistema software**, cioè come deve essere il sistema e sotto quali vincoli deve operare.

Categorie principali

- **Caratteristiche di qualità**: efficienza, affidabilità, safety, usabilità, interfaccia, sicurezza, robustezza, ecc.

- **Caratteristiche del processo di sviluppo**: standard di processo, uso di ambienti CASE, linguaggi di programmazione, metodi di sviluppo, ecc.
- **Caratteristiche esterne**: interoperabilità con sistemi di altre organizzazioni, vincoli legislativi, ecc.
- **Requisiti fisici**: hardware, rete, ecc.

Esempi

- Il **tempo di risposta** del sistema all'inserimento della password utente deve essere **inferiore a 10 secondi**.
- I **documenti di progetto** (deliverable) devono essere **conformi** allo standard *XYZ-ABC-12345*.
- Il sistema software **non deve rilasciare** agli operatori alcuna **informazione personale** relativa ai clienti, tranne nominativo e identificatore.

Requisiti Funzionali e Non-Funzionali

È importante **tenerli separati**, anche quando riguardano la stessa funzionalità, per evitare ambiguità.

Esempio

- Requisito **funzionale**:
  - Il sistema deve validare il **pin inserito dal cliente**.
- Requisito **non-funzionale**:
  - Il sistema deve completare la **validazione del pin entro 3 secondi**.

☝🏻 Si calcola che il 95% delle operazioni di un bancomat venga eseguita tramite
software scritto in Cobol e che in totale ci siano 200 miliardi di righe di codice che

gestiscono ogni giorno 30 miliardi di transazioni.

Descrivere i requisiti di un sistema

Quando si devono descrivere i **requisiti di un sistema software**, esistono due principali approcci:

1. Descrizione basata su **linguaggio naturale**

- **Glossario**: raccolta di termini rilevanti per il dominio, con definizioni chiare per evitare ambiguità.
- **Specifica dei requisiti (funzionali e non-funzionali)**: documento dettagliato che elenca e descrive cosa il sistema deve fare (**requisiti funzionali**) e con quali vincoli e qualità (**requisiti non-funzionali**).

2. Descrizione basata su **linguaggi grafici**

- **Modello del dominio**: rappresentazione grafica di entità, relazioni, processi e comportamenti che caratterizzano il dominio del sistema.
- **Casi d'uso (Use Cases)**: diagrammi e descrizioni che mostrano le interazioni tra gli **attori** (utenti o sistemi esterni) e il sistema, utili per descrivere i **requisiti funzionali**.

👉 Nella pratica, i due approcci **non sono mutuamente esclusivi**: spesso si usano insieme per avere sia una descrizione leggibile (linguaggio naturale) sia una rappresentazione chiara e visuale (grafica).

Documento dei Requisiti

Il **documento dei requisiti** è il risultato principale della fase di analisi dei requisiti.

Contenuti principali

- **Elenca tutti i requisiti del sistema** da realizzare.
- **Specifica cosa deve fare** il sistema (requisiti funzionali) e **quali vincoli** deve rispettare (requisiti non-funzionali).

Ruolo e importanza

- Rappresenta un **contratto formale** tra sviluppatore e committente: stabilisce in modo chiaro cosa dovrà essere implementato.
- Definisce (di norma) anche una **scadenza** per la consegna del prodotto.

Analisi dei Requisiti in 5 Passi

1. **Acquisizione**
   - Raccolta delle esigenze e delle richieste del committente.
2. **Elaborazione**
   - Organizzazione, chiarimento e interpretazione delle informazioni raccolte.
3. **Convalida**
   - Verifica che i requisiti siano corretti, completi e coerenti.
4. **Negoziazione**
   - Risoluzione di conflitti tra requisiti contrastanti e ricerca di compromessi.
5. **Gestione**
   - Monitoraggio continuo e aggiornamento dei requisiti durante l'intero ciclo di vita del progetto.

Acquisizione mediante Interviste

- Gli analisti incontrano i membri dell'organizzazione del committente.
- Le interviste possono essere:
  - **Strutturate** → seguono uno schema preciso di domande.
  - **Non strutturate** → più libere, permettono approfondimenti spontanei.
- **Pro:** consentono un'interazione diretta.
- **Contro:** richiedono esperienza, sono difficili da condurre bene, e le risposte possono essere incomplete o vaghe.

Problema tipico

- Analista: "Cosa vuoi?"
- Cliente: "Progetta il software per me."
- Analista: "Ma cosa vuoi farci?"
- Cliente: "Non so, prima dimmi cosa può fare."
- Analista: "Posso fare qualsiasi cosa..."
- Cliente: "Allora fammi un software che scopra i miei requisiti!"

👉 Questo paradosso mostra come spesso i requisiti non siano chiari fin dall'inizio e vadano scoperti iterativamente.

Acquisizione: Alternative alle Interviste

- **Questionari a risposta multipla** → distribuiti a gran parte dei membri dell'organizzazione.
- **Costruzione di prototipi** → per stimolare il feedback dei futuri utenti.
- **Osservazione diretta** → vedere gli utenti al lavoro per capire i loro bisogni reali.
- **Studio di documenti** → analisi della documentazione esistente dell'organizzazione.

Prototipi

- I prototipi non devono essere necessariamente software funzionante.
- Anche **prototipi su carta** possono essere utili per illustrare interfacce e flussi, e per ottenere feedback iniziale dagli utenti.

Fase di **Elaborazione**

Durante questa fase:

- I requisiti raccolti vengono **espansi, chiariti e raffinati**.
- Si produce la **prima bozza del Documento dei Requisiti**, che fungerà da riferimento per tutte le fasi successive del progetto.

Struttura del Documento dei Requisiti (secondo Sommerville, ispirata allo standard IEEE/ANSI 830-1993)

1. **Introduzione**
   - Spiega *perché* il sistema è richiesto.
   - Mostra come si inserisce negli obiettivi più ampi del committente.
2. **Glossario**
   - Definisce termini tecnici e concetti usati nel documento.
   - Serve a eliminare ambiguità e a mantenere coerenza tra sviluppatori e committente.
3. **Definizione dei requisiti funzionali**
   - Elenco dei servizi che il sistema deve offrire.
   - Descrizione delle funzionalità richieste dagli utenti.
4. **Definizione dei requisiti non-funzionali**
   - Vincoli che riguardano:
     - *Prestazioni* (es. tempi di risposta)

- *Affidabilità*
- *Scalabilità*
- *Sicurezza*
- *Vincoli di processo o tecnologici*.

5. **Architettura del sistema**
   - Suddivisione in sottosistemi o moduli.
   - Ogni requisito deve poter essere ricondotto al sottosistema che lo implementerà.

Struttura del Documento dei Requisiti (cont.)

1. **Specifica di requisiti del software e del sistema**
   - Descrizione dettagliata dei requisiti funzionali.
   - Indica cosa fa il sistema, senza entrare nel *come*.
2. **Modelli astratti del sistema**
   - Diagrammi o modelli formali/semi-formali.
   - Ogni modello rappresenta un punto di vista:
     - **Controllo** → flusso degli eventi.
     - **Dati** → strutture e relazioni.
     - **Funzioni** → trasformazioni e processi.
3. **Evoluzione del sistema**
   - Previsioni su possibili cambiamenti futuri.
   - Esempi: nuove piattaforme hardware, modifiche nei requisiti aziendali, aggiornamenti normativi.
4. **Appendici**
   - Informazioni supplementari come:
     - Specifiche della piattaforma hardware.
     - Requisiti di database.

    - Manuale utente preliminare.
    - Piani di test.
5. **Indici**

- Strumenti di consultazione:
    - *Lemmario* (lista dei termini).
    - Puntatori ai requisiti in cui tali termini compaiono.

Requisiti Ben Posti

Un requisito deve essere **chiaro, specifico e verificabile**.

Esempi corretti:

- “Il sistema deve gestire tutti i registratori di cassa del negozio.”
- “Il sistema deve stampare la sintesi degli incassi della giornata.”
- “A fine giornata, il sistema deve elencare gli articoli da reintegrare in base alle vendite.”
- “Il tempo di risposta del sistema a qualunque interrogazione deve essere inferiore a 3 secondi.”

Documento dei Requisiti: Difetti da Evitare

Quando si scrive un documento dei requisiti, è fondamentale fare attenzione a una serie di difetti comuni che possono compromettere la chiarezza, la completezza e l’utilizzabilità del documento.

Difetti principali:

- **Omissioni (incompletezze)**

  → Mancata presenza di un requisito necessario, quindi il documento risulta incompleto.

- **Inconsistenze**

→ Contraddizioni fra i vari requisiti oppure tra i requisiti e l'ambiente operativo in cui il sistema dovrà essere utilizzato.

- **Ambiguità**

  → Requisiti scritti in maniera tale da poter essere interpretati in più modi diversi.

- **Sinonimi e omonimi**

  → Uso di termini diversi con lo stesso significato (*sinonimi*) o uso di termini uguali con significati differenti (*omonimi*). Questo può generare confusione.

- **Presenza di dettagli tecnici**

  → Nei requisiti non vanno inseriti dettagli implementativi o tecnologici. Il documento deve dire *cosa* fare, non *come*.

- **Ridondanza**

  → In generale va evitata. È accettabile solo tra sezioni diverse del documento (es. glossario e appendici), ma non all'interno della stessa sezione.

Incompletezze

Esempio:

> "Il sistema deve essere in grado di gestire prenotazioni di tavoli."

Questo requisito è **incompleto**, perché non specifica in dettaglio *come* deve avvenire la gestione delle prenotazioni. Bisogna chiarire:

- In che modo gli utenti possono effettuare le prenotazioni (online, telefono, in loco?).
- Quali informazioni sono richieste (nome, numero di persone, data e ora, preferenze alimentari?).
- Come vengono confermate o annullate le prenotazioni?
- È previsto l'**overbooking** (più prenotazioni di quelle effettive disponibili)?
- Sono previsti limiti temporali per prenotare (ad esempio, almeno 2 ore prima)?

Un requisito corretto deve essere **specifico** e coprire i casi principali.

Ambiguità e Vaghezze

Il **linguaggio naturale** è intrinsecamente impreciso, quindi se non controllato può generare molti fraintendimenti.

Esempio:

> "Il sistema deve mostrare TUTTE le strade VICINO alla destinazione."

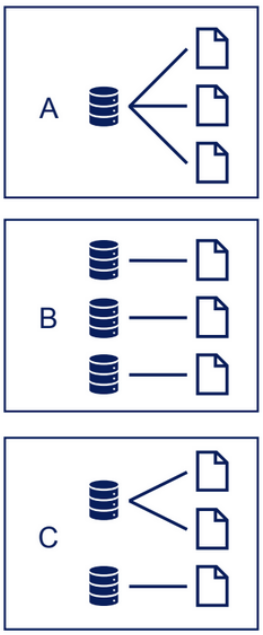
Ambiguità:

- "Vicino" significa entro 100 metri? 1 km?
- Si intendono tutte le strade principali o anche le traverse?

Ambiguità: Quantificatori

Esempio:

> "TUTTI i file sono controllati da UN blocco di controllo."



Possibili interpretazioni:

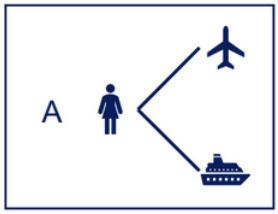
- **A.** Un unico blocco di controllo controlla tutti i file.
- **B.** Ogni file ha un proprio blocco di controllo.
- **C.** Ogni file è controllato da uno specifico blocco di controllo, ma alcuni blocchi possono controllare più file contemporaneamente.

👉 Per ridurre ambiguità, occorre specificare chiaramente il significato dei quantificatori come: *tutti, ogni, sempre, qualsiasi, nessuno*.

Ambiguità: Disgiunzioni

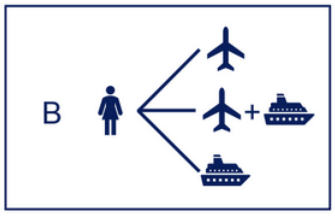
Esempio:

> “Viaggerò in aereo o in traghetto.”



Interpretazioni possibili:

- **A. XOR (mutuamente esclusivo)** → o aereo o traghetto, ma non entrambi.
- **B. OR (debole, inclusivo)** → posso viaggiare in aereo, in traghetto, o in entrambi.

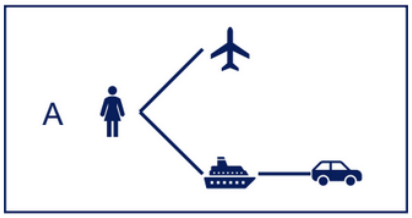


👉 Ogni volta che si usa “o” in un requisito, bisogna chiarire se è **esclusivo** o **inclusivo**.

Ambiguità di Coordinazione

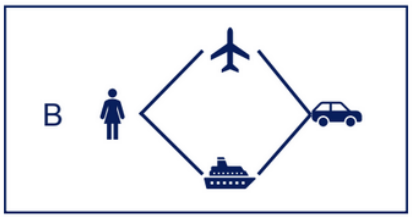
Esempio:

> “Viaggerò in aereo o in traghetto e in macchina.”



Interpretazioni possibili:

- **A.** Viaggerò in aereo, oppure viaggerò in traghetto e in macchina.
- **B.** Viaggerò in aereo o in traghetto, e comunque anche in macchina.



👉 Per evitare fraintendimenti è meglio spezzare la frase in più requisiti chiari.

Ambiguità Referenziale

Esempio:

> “Per quelli di voi che hanno bambini e non lo sanno, abbiamo un asilo al piano di sotto.”

Interpretazioni possibili:

- **A.** Non sanno di avere bambini (!).
- **B.** Non sanno che c’è un asilo al piano di sotto.

👉 L’ambiguità referenziale si presenta quando non è chiaro a cosa si riferisca un **pronome** (lo, la, esso, loro, ecc.).

Ambiguità Anaforica

Esempio:

> “La procedura convertirà l'immagine a 24 bit in un'immagine a 8 bit, quindi la mostrerà in una finestra dinamica.”

Interpretazioni possibili:

- **A.** Mostra l’immagine originale a 24 bit.
- **B.** Mostra l’immagine convertita a 8 bit.

👉 Per evitarlo, sostituire il pronome con il sostantivo esplicito.

Ambiguità: Vaghezze

Esempio:

> “Il sistema software deve fornire un appropriato visualizzatore per i documenti memorizzati.”

“Appropriato” è un termine **vago**:

- Cosa significa appropriato?
- Deve essere veloce? Con interfaccia grafica? Con zoom?

👉 Cercare **aggettivi e avverbi non misurabili** (appropriato, veloce, semplice, vantaggioso, accessibile, brevemente, ecc.). Sono da evitare, oppure devono essere sostituiti con metriche precise.

Verbi Deboli e Forme Passive

Esempio:

> “Il sistema deve includere allarmi sonori, che possono essere attivati quando viene rilevata una violazione della sicurezza da un contatto magnetico.”

Ambiguità:

- Chi attiva l’allarme? L’utente? Il sistema automaticamente?

👉 Da evitare:

- **Verbi deboli** come *potere*.
- **Forme passive** senza complemento di agente.

Scrivere sempre in forma attiva e con soggetto esplicito.

Non usare (doppie) Negazioni

Le negazioni rendono i requisiti meno chiari.

Esempio (negazione singola):

> “Gli utenti con tre o più account non devono essere migrati.”
>
> ✔️ Migliore: “Il sistema deve migrare solo gli utenti con meno di tre account.”

Esempio (doppia negazione):

> “I premi e i punti non saranno visibili agli utenti che non partecipano al concorso.”
>
> ✔️ Migliore: “I premi e i punti saranno visibili solo agli utenti che partecipano al concorso.”

👉 Le frasi con negazioni sono più difficili da interpretare, soprattutto per un documento tecnico che deve essere letto in modo univoco.

Tecniche di convalida dei requisiti

La convalida serve a verificare che i requisiti raccolti siano:

- corretti,
- completi,
- non ambigui,
- consistenti tra loro,
- effettivamente utili al cliente e al sistema da sviluppare.

Per farlo si usano varie tecniche:

1. **Deskcheck**
    - Revisione informale del documento dei requisiti, fatta da uno sviluppatore o analista singolo.
    - Serve a trovare errori di scrittura, incoerenze o parti poco chiare.
    - È un primo controllo “a tavolino”.
2. **Walkthrough**
    - Lettura sequenziale del documento insieme a più persone (committente, analisti, sviluppatori).
    - L’obiettivo è camminare passo passo dentro il documento per discutere ogni requisito.
    - È utile per scoprire ambiguità o punti non chiari in modo collaborativo.
3. **Ispezione**
    - Revisione più strutturata e formale.
    - Si usano tecniche di lettura mirata, come:
        - **Tecnica del lemmario**: controllare sistematicamente le parole che possono generare ambiguità (es. “adeguato”, “appropriato”, “rapido”).
        - Ricerca di **rimozioni, distorsioni, generalizzazioni** (concetti introdotti da Chomsky nel 1975 per analizzare la lingua naturale).
            - **Rimozioni** → informazioni mancanti.

        - **Distorsioni** → formulazioni che cambiano il significato originario.
        - **Generalizzazioni** → requisiti troppo vaghi, che lasciano spazio a interpretazioni multiple.

4. **Uso di strumenti di analisi del linguaggio naturale (NLP)**
    - Software che analizzano automaticamente i testi e segnalano possibili ambiguità o incoerenze.
    - Molto utile in sistemi complessi o con requisiti molto numerosi.
5. **Prototipi**
    - Creazione di una versione preliminare del sistema (anche molto semplificata) per far vedere al cliente **come funzionerebbe**.
    - Serve a chiarire i requisiti e individuare incongruenze o mancanze.
    - È una delle tecniche più efficaci, perché fa emergere i problemi quando gli utenti vedono "qualcosa di concreto".

Esempi di convalida di requisiti

- **R1: "Il sistema software deve fornire un appropriato visualizzatore per i documenti memorizzati"**
    - Problema: "appropriato" è un termine **vago**, ognuno potrebbe interpretarlo diversamente.
    - Il requisito diventa ambiguo. Bisogna definire: appropriato in che senso? Supporta determinati formati? Ha funzioni di zoom? Evidenziazione?
- **R2: "L'utente deve essere in grado di effettuare ricerche sia sull'intero insieme di basi di dati che su un loro sottoinsieme"**
    - Problema: "un loro sottoinsieme" è **ambiguo** → significa uno solo? Più di uno? Quali criteri di selezione?
    - Il resto è ok: requisito espresso dal punto di vista dell'utente, stile caso d'uso.
- **R3: "Ad ogni nuovo ordine deve essere associato un identificatore unico (Order_ID)"**

- Problema: requisito di **basso livello tecnico** → introduce vincoli implementativi (design).
- Domanda: è necessario all'utente? O è una scelta tecnica che riguarda solo lo sviluppatore?
- Meglio riformularlo come vincolo progettuale, non come requisito utente.

Convalida mediante tecnica del lemmario

- Si costruisce un **glossario dei termini** usati nel documento dei requisiti.
- Ogni termine presente nel glossario viene collegato ("puntato") ai requisiti che lo utilizzano.

🔎 Questo serve a:

1. **Individuare inconsistenze**
   - Se lo stesso termine è usato con significati diversi in requisiti differenti, c'è un problema.
   - Esempio: "utente" in un requisito indica un amministratore, in un altro un cliente → inconsistenza.
2. **Scoprire sinonimi**
   - Stesso concetto espresso con termini diversi.
   - Esempio: "login" e "autenticazione" usati in requisiti diversi per dire la stessa cosa → conviene uniformare.
3. **Scoprire omonimi**
   - Stesso termine con significati diversi.
   - Esempio: "ordine" come "ordine di acquisto" e "ordine" come "ordine cronologico".
4. **Individuare ridondanze**
   - Requisiti che dicono la stessa cosa con parole diverse.

- Esempio: R5 “Il sistema deve consentire all’utente di autenticarsi” e R7 “Il sistema deve permettere il login all’utente” → ridondanza.

👉 In sostanza, il glossario diventa uno strumento di **tracciabilità semantica**, cioè permette di controllare che ogni termine sia usato in modo coerente in tutto il documento.

Convalida mediante NLP (Natural Language Processing)

- Si applicano strumenti software che fanno analisi **lessicale** e **sintattica** sul documento dei requisiti.
- Possono segnalare automaticamente:
  - parole ambigue o vaghe (es. “rapido”, “facile”, “sicuro” → senza misura sono ambigue);
  - frasi troppo lunghe o complesse che rendono difficile la comprensione;
  - incoerenze tra requisiti (uso diverso di termini o strutture);
  - ridondanze tra requisiti.

📌 Esempio pratico:

- Un tool NLP trova che nel documento i termini “utente”, “customer”, “client” e “utilizzatore” vengono usati tutti per lo stesso concetto → suggerisce di uniformarli.
- Oppure rileva che il requisito R1 contiene l’aggettivo “appropriato”, che è un termine **soggettivo** → quindi lo segnala come ambiguità.

👉 L’uso combinato di:

1. **Glossario con puntatori ai requisiti**
2. **Strumenti NLP per l’analisi linguistica**

permette di ridurre drasticamente i rischi di incomprensione tra committente e sviluppatore, aumentando la qualità del documento dei requisiti.

Gli strumenti NLP vengono usati per **analizzare automaticamente i requisiti espressi in linguaggio naturale**, individuando errori, ambiguità e ridondanze.

Ecco alcuni strumenti specifici:

- **QuARS (Quality Analyzer for Requirements Specifications)**
  - Sviluppato dall'**ISTI-CNR di Pisa**.
  - È gratuito.
  - Analizza testi di requisiti e segnala frasi ambigue, troppo complesse o con termini vaghi.
- **TIGER-PRO**
  - Strumento accademico (anch'esso free).
  - Fornisce supporto per la valutazione della qualità dei requisiti, in particolare nella fase di validazione.
- **Requirements Scout (Qualicen GmbH)**
  - Strumento industriale.
  - Permette di definire regole personalizzate di analisi linguistica e di coerenza.
- **QVscribe (QRA Corp.)**
  - Molto usato in ambito industriale.
  - Integra plugin per MS Word ed Excel.
  - Evidenzia automaticamente requisiti ambigui, incompleti o non verificabili.
- **RAT (Reuse Company)**
  - "Requirements Authoring Tool".
  - Supporta la scrittura di requisiti di qualità, applicando regole sintattiche e semantiche.
- **RQA (IBM)**
  - Strumento più avanzato e integrato in suite enterprise.
  - Analizza e gestisce requisiti, assicurando qualità e tracciabilità.

📌 Questi strumenti non sostituiscono l'analista, ma sono un **supporto**: segnalano criticità che poi devono essere discusse e risolte con competenza umana.

Convalida mediante prototipi

Un altro approccio alla validazione dei requisiti è costruire **prototipi del sistema**.

- **Definizione**: il prototipo è una versione semplificata, parziale o simulata del sistema che serve a chiarire meglio i requisiti.
- **Scopo**: non è il prodotto finale, ma uno strumento per discutere con il cliente e gli utenti finali.

Tipi di prototipi

1. **Prototipi a bassa fedeltà**
   - Es. schizzi su carta, wireframe, mockup.
   - Molto veloci da produrre.
   - Servono a chiarire idee e flussi senza dettagli tecnici.
2. **Prototipi ad alta fedeltà**
   - Versioni software funzionanti ma incomplete.
   - Possono simulare funzioni chiave, interfacce o interazioni con l'utente.
   - Più costosi da sviluppare ma più vicini al prodotto reale.

Vantaggi

- Permettono al committente e agli utenti di **vedere e provare** qualcosa di concreto.
- Aiutano a scoprire:
  - requisiti mancanti,
  - requisiti ambigui,
  - aspettative non realistiche.
- Facilitano la comunicazione tra analisti e stakeholder.

Limiti

- Possono dare un'**illusione di completezza** ("se funziona il prototipo allora il sistema è pronto" → falso).
- Richiedono comunque tempo e risorse per essere sviluppati.

Sintesi comparativa

Aspetto	NLP	Prototipi
Focus	Qualità linguistica dei requisiti scritti	Esperienza d'uso, interazione e funzionalità
Automazione	Alta (analisi automatica)	Bassa (sviluppo manuale)
Coinvolgimento utenti	Indiretto	Diretto
Errori rilevati	Ambiguità, inconsistenze, vaghezze	Requisiti mancanti, usabilità, aspettative
Costo/tempo	Basso/medio	Medio/alto
Uso ideale	Grandi documenti formali di requisiti	Progetti user-centered, interfacce, UX

Negoziazione dei requisiti

Durante la fase di **negoziazione** dei requisiti si entra in un momento cruciale dell'analisi, in cui non si tratta più soltanto di raccogliere e descrivere cosa il sistema deve fare, ma di **bilanciare esigenze, vincoli e risorse disponibili**.

L'obiettivo è assegnare **priorità** ai requisiti tenendo conto di due fattori principali:

- **Le esigenze del committente**: quali funzionalità o caratteristiche sono percepite come più importanti, necessarie o strategiche per il successo del progetto.
- **L'analisi di costi e tempi di produzione**: alcuni requisiti, pur essendo desiderabili, possono risultare troppo onerosi in termini di sforzo di sviluppo, complessità tecnica o tempistiche, e quindi vanno ridimensionati o posticipati.

In questa fase, le priorità stabilite diventano uno strumento pratico per:

- **cancellare** alcuni requisiti non essenziali (che aggiungono complessità senza reale valore per il cliente),
- oppure decidere di **svilupparli in un secondo momento**, seguendo un approccio a **ciclo di vita incrementale** (tipico dei modelli agili o di release successive).

La tecnica MoSCoW per la negoziazione

Un metodo molto diffuso per gestire la priorità dei requisiti è la tecnica **MoSCoW**, che suddivide i requisiti in quattro categorie:

1. Must have

- Sono i requisiti **obbligatori** e irrinunciabili.
- Senza di essi il sistema non sarebbe accettabile dal punto di vista del committente.
- Rappresentano il **core** del prodotto e vengono sempre inclusi nella prima versione del sistema.
- Esempio: “Il sistema deve permettere agli utenti di autenticarsi con username e password.”

2. Should have

- Sono i requisiti **molto importanti**, ma **non strettamente necessari** per la prima versione del sistema.
- Aggiungono valore e migliorano l’esperienza utente, ma in caso di vincoli stringenti (tempo/costi) possono essere rimandati.
- Esempio: “Il sistema dovrebbe permettere l’autenticazione anche tramite social login.”

3. Could have

- Requisiti **opzionali**, di utilità relativa.

- Possono essere implementati se rimane tempo e budget, ma non compromettono la qualità complessiva del sistema se non presenti.
- Esempio: “Il sistema potrebbe includere un tema grafico personalizzabile per ogni utente.”

4. Want to have (o Would like to have)

- Sono i requisiti **differibili** e destinati quasi sempre a versioni future.
- Spesso rappresentano desideri del committente che non incidono sull’accettabilità del prodotto iniziale.
- Sono **contrattabili** e possono essere usati come “moneta di scambio” nelle trattative con il cliente.
- Esempio: “Il sistema, in una versione futura, dovrebbe offrire un assistente virtuale basato su intelligenza artificiale.”

Perché è importante la negoziazione

La negoziazione dei requisiti è fondamentale per:

- evitare conflitti tra **stakeholder** con priorità diverse,
- mantenere il progetto **realistico** rispetto a risorse e tempi,
- garantire che la prima versione del sistema sia già **utilizzabile e accettabile**,
- pianificare in modo ordinato gli sviluppi successivi.

Gestione dei requisiti: identificazione, attributi, tracciabilità e aspetti contrattuali

Identificazione dei requisiti

Nella **gestione dei requisiti** un punto fondamentale è la **corretta identificazione** di ciascun requisito.

Ogni requisito deve avere un **identificatore univoco**, così da poter essere facilmente richiamato, discusso, modificato o verificato.

Gli identificatori possono seguire diverse convenzioni, ad esempio:

- **Numero sequenziale semplice**: 1, 2, 3, ... (semplice ma poco adatto a documenti complessi).
- **Numerazione strutturata**: 2.4.7 (richiama la posizione del requisito all'interno della struttura gerarchica del documento dei requisiti).
- **Coppia <categoria, numero>**: ad esempio *FUN-01* per un requisito funzionale, *NF-05* per un requisito non-funzionale. Questo approccio è utile per distinguere facilmente le categorie e rende la gestione più chiara.

L'identificatore deve essere **stabile nel tempo**, anche se il documento viene modificato. Non deve cambiare se il requisito rimane lo stesso.

Utilizzo di attributi per i requisiti

Oltre all'identificatore, a ogni requisito possono essere associati **attributi aggiuntivi** che ne descrivono meglio la natura e lo stato.

Esempi di attributi tipici:

- **Stato**:
  - Proposto (ancora in discussione)
  - Approvato (accettato dal committente)
  - Rifiutato (scartato in fase di negoziazione)
  - Incorporato (già realizzato e integrato nel sistema)
- **Priorità**: definisce l'importanza del requisito (es. con la tecnica **MoSCoW**: Must, Should, Could, Want).
- **Effort (sforzo)**: stima del lavoro necessario, misurato ad esempio in **giorni/uomo** o punti storia (in ottica agile).

- **Rischio**: misura la difficoltà tecnica o l'incertezza nell'implementazione. Requisiti ad alto rischio vanno monitorati più da vicino.
- **Stabilità**: indica quanto è probabile che il requisito cambi nel tempo.
  - Un requisito "stabile" è ben definito e poco soggetto a modifiche.
  - Un requisito "instabile" potrebbe variare a seconda dell'evoluzione del dominio o delle esigenze del cliente.
- **Versione di destinazione**: utile nello sviluppo **incrementale** o agile, per sapere in quale release/versione del sistema quel requisito verrà implementato.

Questo approccio arricchisce la gestione e rende possibile filtrare, ordinare e monitorare i requisiti in modo sistematico.

Tracciabilità dei requisiti

La **tracciabilità** è la capacità di seguire e descrivere l'intero ciclo di vita di un requisito.

Significa poter stabilire un legame tra ciascun requisito e gli artefatti del processo di sviluppo.

In particolare, ogni requisito può (e deve) essere tracciato rispetto a:

- **Componenti del sistema** che lo realizzano (architettura e design).
- **Codice** che implementa quel requisito.
- **Test** che ne verificano il corretto funzionamento.

La tracciabilità è fondamentale per:

- garantire che **nessun requisito venga dimenticato**;
- facilitare la **manutenzione**: se un requisito cambia, è possibile capire quali componenti, moduli o test devono essere aggiornati;
- gestire i **casi di verifica** e validazione, mostrando la corrispondenza tra requisiti e casi di test.

Per supportare questa attività esistono strumenti CASE (Computer-Aided Software Engineering), come ad esempio **IBM DOORS, Jama, Polarion**, che offrono funzionalità avanzate per la gestione e tracciabilità dei requisiti.

Requisiti e aspetti contrattuali

Il documento dei requisiti ha anche un forte valore legale e contrattuale. Infatti:

- Normalmente **precede la stipula del contratto** e ne diventa parte integrante: stabilisce cosa lo sviluppatore deve realizzare e cosa il committente deve aspettarsi.
- Se al momento della stipula **non è possibile avere un documento definitivo**, è opportuno prevedere nel contratto una clausola di **rinegoziazione**, così da poter aggiustare i dettagli man mano che i requisiti vengono chiariti.
- Nei casi di **gara o appalto**, il documento dei requisiti può essere prodotto direttamente dal **committente**, come parte di un **capitolato tecnico** a cui i fornitori devono attenersi.

CASI D'USO

I **casi d'uso** (Use Cases) sono una tecnica classica nell'ingegneria dei requisiti, molto utilizzata in contesti **plan-driven** ma anche in ambienti agili più strutturati.

Un caso d'uso descrive un **modo in cui un utente interagisce con il sistema** per raggiungere un obiettivo specifico.

- Il "protagonista" è l'**attore** (ad esempio: utente, amministratore, sistema esterno).
- L'interazione è descritta come una **sequenza di eventi**.

Elementi chiave

- **Scenario principale**: la sequenza "ideale" (flusso base) che porta al risultato desiderato.
- **Scenari alternativi / eccezioni**: comportamenti inattesi, errori, deviazioni dal flusso principale (es. input errati, rete non disponibile, pagamento non riuscito).
- **Risultato atteso**: cosa ottiene l'attore al termine dell'interazione.

Caratteristiche

- Linguaggio narrativo, comprensibile anche al committente.
- Possono essere accompagnati da **diagrammi UML dei casi d'uso**, che mostrano attori e relazioni con i casi d'uso.
- Utile per la comunicazione con il cliente: si prospetta lo scenario e si attende il feedback.

USER STORIES

Le **user stories** nascono nei contesti **Agile** (XP, Scrum, ecc.).

Sono **brevi descrizioni** dei requisiti, sempre dal punto di vista dell'utente, con un **formato standardizzato**:

```
As a <ruolo>
I want <obiettivo>
so that <beneficio>
```

Esempio:

"In qualità di cliente voglio cancellare la mia prenotazione in hotel, per avere un rimborso."

USER STORY CARDS

Le user stories tradizionalmente vengono scritte su **schede fisiche (A6)** per mantenere la sintesi e favorire la collaborazione visiva del team.

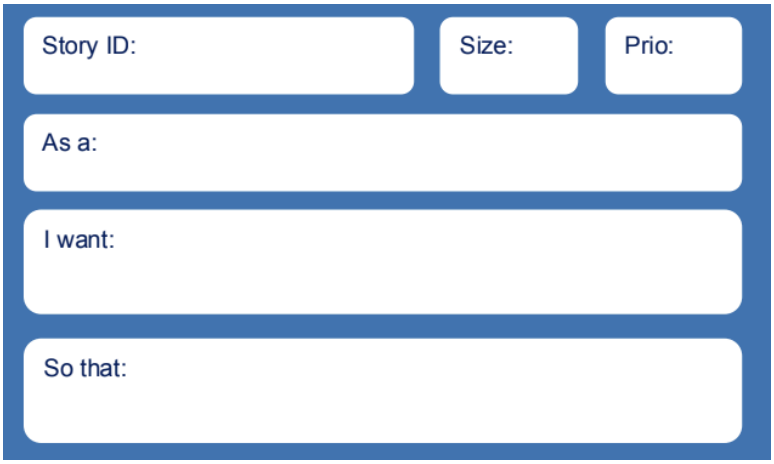
Struttura tipica:

- **Story ID** (identificatore)
- **Size** (dimensione/stima)
- **Prio** (priorità)
- **As a ...**
- **I want ...**

- **So that ...**

Vantaggi delle card:

- Facilità di manipolazione fisica.
- Possibilità di **raggrupparle e ordinarle** su un muro o tavolo.
- Favoriscono la **visibilità** e la discussione collettiva.



LIMITAZIONI DELLE USER STORIES

Nonostante la loro efficacia, le user stories presentano dei limiti, soprattutto in contesti complessi:

1. **Scalabilità**
   - Le schede fisiche diventano ingestibili su grandi progetti o team distribuiti.
   - La gestione digitale aiuta (es. Jira, Trello, Azure DevOps).
2. **Vaghezza / informalità**
   - Sono pensate come **spunti di conversazione**, non come requisiti esaustivi.
   - Non adatte per **contratti legali** o documentazione formale.
3. **Incompleti dettagli**
   - Non riportano tutti i dettagli tecnici, specialmente per i **requisiti non funzionali** (prestazioni, sicurezza, affidabilità).

- Spesso necessitano di ulteriori strumenti (Acceptance Criteria, Definition of Done, backlog refinement).